

Implementação de artigos de segmentação de imagens

Bruno Braga Guimarães Alves¹

João Vítor de Freitas Scarlatelli¹

Nagib Alexandre Verly Borjaili¹

Vitor Dias de Britto Militão¹

Yasmin Cassemiro Viegas¹

¹ Pontifícia Universidade Católica de Minas Gerais
Ciência da Computação

Abstract. *This article presents the implementation and analysis of two distinct image segmentation methods based on graph theory, as described in the works of [Brown et al. 2013] and [Boykov and Funka-Lea 2006]. The first method proposes a hierarchical clustering approach based on minimizing internal and external differences between clusters, allowing the extraction of a perceptual structure from the image. The second method employs the concept of maximum flow/minimum cut to model segmentation as an optimization problem, efficiently dividing the image into foreground and background regions. Both methods are widely used due to their efficiency and robustness in various computer vision applications, and this work aims to explore their characteristics and compare their performance.*

Resumo. *Este artigo apresenta a implementação e a análise de dois métodos distintos de segmentação de imagens baseados na teoria de grafos, descritos nos trabalhos de [Brown et al. 2013] e [Boykov and Funka-Lea 2006]. O primeiro método propõe uma abordagem de agrupamento hierárquico, baseada na minimização das diferenças internas e externas entre os agrupamentos, permitindo a extração de uma estrutura perceptual da imagem. Já o segundo método utiliza o conceito de fluxo máximo/corte mínimo para modelar a segmentação como um problema de otimização, separando eficientemente a imagem em regiões de foreground e background. Ambos os métodos são amplamente utilizados devido à sua eficiência e robustez em diferentes aplicações de visão computacional, e este trabalho busca explorar suas características e comparar seus desempenhos.*

Participação

A realização deste trabalho contou com a colaboração ativa dos seguintes autores:

- **Bruno Braga Guimarães Alves:** Implementação Referente ao Artigo 1 - "Efficient Graph Segmentation"
- **João Vítor de Freitas Scarlatelli:** Implementação Referente ao Artigo 1 - "Efficient Graph Segmentation"
- **Nagib Alexandre Verly Borjaili:** Implementação Referente ao Artigo 2 - "Graph Cuts and Efficient N-D Image Segmentation"
- **Vitor Dias de Britto Militão:** Implementação Referente ao Artigo 1 - "Efficient Graph Segmentation"
- **Yasmin Cassemiro Viegas:** Implementação Referente ao Artigo 2 - "Graph Cuts and Efficient N-D Image Segmentation"

Primeiro método - Efficient Graph Segmentation

Introdução

Este artigo descreve a implementação de um algoritmo para segmentação de imagens utilizando o método descrito por Felzenszwalb e Huttenlocher (2004). Este método propõe uma abordagem eficiente baseada na teoria de grafos, modelando a segmentação como um problema de agrupamento. A técnica utiliza o conceito de minimização de diferenças internas e externas entre os agrupamentos para criar uma hierarquia de regiões que reflete a estrutura perceptual da imagem.

Estrutura do Código

1. Arquivo `filter.cpp`: Pré-processamento da Imagem

Propósito

Realiza suavização (redução de ruído) e calcula a diferença de cores entre pixels, preparando os dados para a segmentação.

Componentes Principais

- **Classe `Filter`**: Contém as funcionalidades principais para manipulação de imagens.
 - **`GaussianMask`**: Cria uma máscara gaussiana usada para suavização.
 - **`Image`**: Representa a imagem como uma matriz unidimensional.
- **Métodos principais**:
 - `smooth`: Aplica a máscara gaussiana para suavizar os canais R, G e B.
 - `diff`: Calcula a diferença de cor entre dois pixels.

2. Arquivo `segment_graph.cpp`: Segmentação Baseada em Grafos

Propósito

Constrói um grafo representando os pixels da imagem e realiza a segmentação em regiões conectadas.

Componentes Principais

- **Classe `Edge`**: Representa uma aresta no grafo com vértices *a*, *b* e peso *w*.
- **Classe `Universe`**: Implementa o algoritmo de *Union-Find* para agrupar pixels.
- **Função `segment_graph`**:
 - Ordena as arestas por peso.
 - Une conjuntos de pixels se a diferença for menor que um limiar dinâmico.
 - Atualiza os limiares com base no tamanho dos conjuntos.

3. Arquivo `main.cpp`: Controle Principal

Propósito

Gerencia o fluxo do programa, integrando as diferentes partes e controlando entrada e saída de dados.

Estrutura

- **Conversão de formatos:** Usa os scripts `pngToPpm.py` e `ppmToPng.py` para converter entre PNG e PPM.
- **Funções de manipulação de arquivos:**
 - `read_ppm`: Lê uma imagem no formato PPM.
 - `write_ppm`: Salva uma imagem processada no formato PPM.
- **Pipeline de segmentação:**
 - Pré-processamento: Aplica suavização.
 - Construção do grafo: Representa pixels e suas diferenças como arestas.
 - Segmentação: Chama `segment_graph` para criar regiões conectadas.
 - Pós-processamento: Une regiões pequenas a outras maiores.
- **Parâmetros configuráveis:**
 - `sigma`: Suavização.
 - `k`: Controle de segmentação.
 - `min_size`: Tamanho mínimo das regiões.

4. Arquivo `pngToPpm.py`

Propósito

Este script Python converte arquivos no formato PNG para PPM, que é mais fácil de manipular diretamente no código C++.

5. Arquivo `ppmToPng.py`

Propósito

Este script Python converte arquivos no formato PPM para PNG, que é mais fácil de manipular diretamente no código C++.

Como Funciona o fluxo geral

1. **Entrada:** A imagem é fornecida em formato PNG.
2. **Conversão para PPM:** O script `pngToPpm.py` transforma o arquivo.
3. **Pré-processamento:** O `filter.cpp` suaviza a imagem e calcula as diferenças entre pixels.
4. **Construção e segmentação do grafo:** O arquivo `segment_graph.cpp` cria o grafo e realiza a segmentação.
5. **Conversão de saída:** O resultado em PPM é transformado novamente para PNG pelo script `ppmToPng.py`.
6. **Saída:** O programa salva a imagem segmentada no diretório especificado.

Estrutura de Dados Utilizada

1. Representação da Imagem

- **Estrutura:** A imagem é representada pela estrutura `Image`.
- **Detalhes:**
 - `width` e `height` armazenam as dimensões da imagem.
 - Um vetor unidimensional (`std::vector<RGB>`) contém os pixels da imagem, onde cada pixel é representado pela estrutura `RGB`, que possui os canais de cor `r`, `g`, e `b`.
- **Justificativa:** A utilização de um vetor unidimensional facilita o acesso sequencial aos pixels e permite o mapeamento direto de coordenadas bidimensionais (`x`, `y`) para índices lineares no vetor.

2. Representação do Grafo

- **Estrutura:** Vetor de arestas (`std::vector<Edge>`)
- **Detalhes:**
 - Cada aresta conecta dois pixels (vértices) adjacentes e possui um peso baseado na diferença de cor entre eles.
 - As conexões consideradas são:
 1. Direita
 2. Abaixo
 3. Diagonal inferior direita
 4. Diagonal superior direita
- **Justificativa:** Essa representação é eficiente e permite o uso direto de algoritmos baseados em teorias gráficas, como a Árvore Geradora Mínima (MST).

3. Estrutura de Conjuntos Disjuntos (Union-Find)

- **Estrutura:** `Universe`
- **Detalhes:**
 - Implementa `Union-Find` com otimizações de compressão de caminho e união por classificação.
 - Cada componente representa um segmento da imagem.
 - Funções principais:
 - * `find(x)` : Encontra o representante de um conjunto.
 - * `join(a, b)` : Une dois conjuntos.
 - * `size(a)` : Retorna o tamanho do componente.
- **Justificativa:** O `Union-Find` é ideal para problemas de agrupamento devido à eficiência em operações repetitivas de união e busca, especialmente em grafos densos.

4. Filtros de Suavização

- **Estrutura:** Vetores de canais RGB (`std::vector<float>`)
- **Detalhes:**
 - Cada canal de cor (R, G, B) é separado em vetores distintos.
 - A suavização é realizada por meio de um filtro gaussiano, reduzindo o ruído e melhorando a consistência na atribuição dos pesos das arestas.
- **Justificativa:** A suavização evita a criação de segmentos desnecessários causados por flutuações abruptas em pequenas regiões.

5. Vetores de Cores Aleatórias

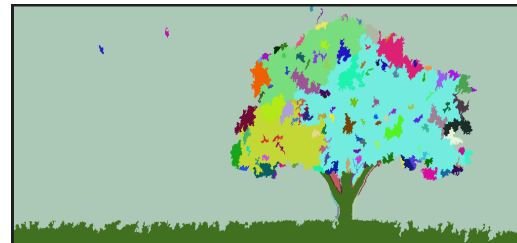
- **Estrutura:** `std::vector<RGB>`
- **Detalhes:**
 - Cada componente segmentado recebe uma cor distinta, gerada aleatoriamente pela função `random_rgb`.
- **Justificativa:** As cores distintas ajudam a visualizar os diferentes segmentos de forma clara na imagem final.

Melhoria na Segmentação

- Após a segmentação inicial, componentes pequenos (abaixo de um limite de tamanho `min_size`) são fundidos com componentes maiores adjacentes, para evitar segmentos irrelevantes ou causados por ruído.
- Este processo é realizado iterativamente utilizando o `Union-Find`, garantindo eficiência no pós-processamento.

Geração da Imagem Segmentada:

Cada componente segmentado é colorido com uma cor distinta. No final, o código salva a imagem segmentada no formato PPM e converte-a de volta para PNG, pronta para visualização.



Segundo Método - Graph Cuts and Efficient N-D Image Segmentation

Introdução - Graph Cuts and Efficient N-D Image Segmentation

Este artigo descreve a implementação de um algoritmo para **segmentação de imagens** utilizando o método de *Graph Cuts*, baseado nos conceitos apresentados no artigo [Boykov and Funka-Lea 2006]. Este método modela o problema de segmentação como um fluxo máximo/corte mínimo em um grafo, separando a imagem em regiões (*foreground* e *background*) de maneira eficiente.

Estrutura do Código

O código é organizado em funções e classes principais, descritas a seguir.

1. Classe `Graph`

A classe `Graph` representa o grafo utilizado para modelar o problema de segmentação. Contém:

- **Arestas (`Edge`):** Representadas por destino, capacidade e fluxo.
- **`addEdge`:** Adiciona arestas bidirecionais entre dois nós, permitindo cálculos de capacidade residual.
- **`bfs`:** Implementa uma busca em largura (*BFS*) para encontrar caminhos aumentantes no grafo. Avalia arestas com capacidade residual positiva.
- **`edmondsKarp`:** Aplica o algoritmo Edmonds-Karp para determinar o fluxo máximo e o corte mínimo, atualizando os fluxos nas arestas e somando o fluxo total.

2. Funções para Construção do Grafo

`computeTLinks`

- Calcula os **t-links** com base no histograma da imagem.
- Utiliza o **método de Otsu** para determinar o limiar de intensidade, identificando quais pixels são mais propensos a pertencer ao *foreground* ou ao *background*.
- **Estrutura de Dados:** Armazena os custos de ligação nos vetores `sourceCost` e `sinkCost`, que são usados na construção do grafo.

`computeNLinks`

- Adiciona **n-links** entre pixels vizinhos.
- Os pesos são calculados com a função `computeEdgeWeight`, que utiliza uma métrica Gaussiana para medir similaridade entre pixels com base na diferença de intensidade.
- **Estrutura de Dados:** As conexões são representadas como arestas no grafo usando a lista de adjacências.

3. Funções para Manipulação de Imagens

`readPPM`

- Lê uma imagem no formato PPM binário (RGB) e a carrega em uma estrutura tridimensional ($\text{altura} \times \text{largura} \times \text{canais}$).
- Verifica o formato e extrai as dimensões e os valores dos pixels.
- **Estrutura de Dados:** Utiliza uma matriz tridimensional (`vector<vector<vector<int>>>`) para representar os valores dos pixels.

`savePPM`

- Salva uma imagem segmentada no formato PPM binário.
- Pixels são coloridos como *foreground* (preto) ou *background* (branco) com base nos resultados do corte.
- **Estrutura de Dados:** Também utiliza uma matriz tridimensional para armazenar os valores dos pixels.

4. Algoritmo Principal

O algoritmo principal segue os seguintes passos:

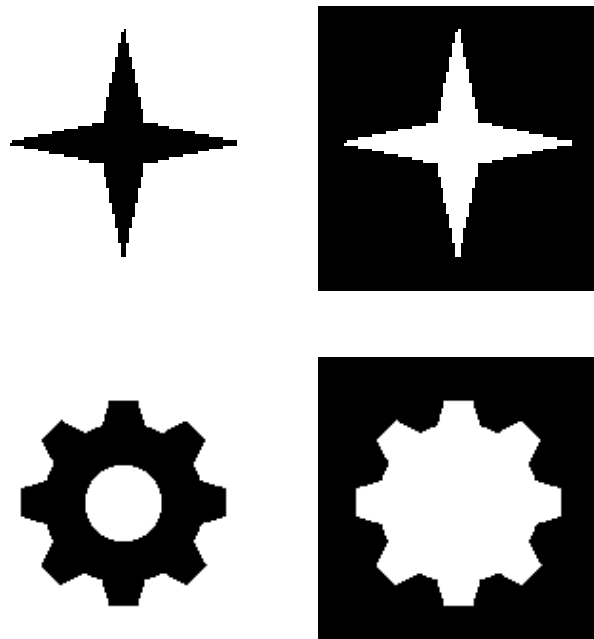
1. **Leitura da Imagem:** A imagem original é carregada com `readPPM`.
2. **Construção do Grafo:**
 - `computeTLinks` calcula os custos de ligação (**t-links**).
 - `computeNLinks` adiciona conexões entre pixels vizinhos (**n-links**).
 - `addEdge` adiciona as conexões ao grafo.
3. **Estrutura de Dados:** O grafo é representado como uma lista de adjacências, onde cada nó contém as arestas conectadas a ele.

4. **Cálculo do Corte Mínimo:** O método `edmondsKarp` calcula o fluxo máximo e identifica o corte mínimo no grafo.
5. **Geração da Imagem Segmentada:**
 - Os nós conectados ao *source* após o corte são atribuídos ao *foreground*.
 - Os demais são atribuídos ao *background*.
6. **Saída:** A imagem segmentada é salva com `savePPM`, e o fluxo máximo é exibido.

Resultados e Considerações

A implementação realiza a segmentação de imagens no formato PPM, separando objetos em *foreground* e *background*. Assim como a entrada, o resultado segue em formato PPM que optamos por ser mais eficiente para tratar em código. Incluímos durante o processo dois arquivos em Python somente para a conversão de formatos, de PNG para PPM e vice-versa; a implementação do artigo na prática segue totalmente em linguagem C++.

Testamos em alguns modelos de imagens, destacando os resultados em duas delas: a imagem de uma engrenagem (*imagem1*) e a imagem de uma estrela (*imagem2*).



References

- [Boykov and Funka-Lea 2006] Boykov, Y. and Funka-Lea, G. (2006). Graph cuts and efficient n-d image segmentation. *International Journal of Computer Vision*, 70(2):109–131. Disponível em: <https://www.csd.uwo.ca/~yboykov/Papers/ijcv06.pdf>. Acesso em: 15 dez. 2024.
- [Brown et al. 2013] Brown, M., Felzenszwalb, P. F., and Dickinson, S. (2013). Sparse representations for image segmentation. *International Journal of Computer Vision*, 102(1-3):88–111. Disponível em: <https://cs.brown.edu/people/pfelzens/papers/seg-ijcv.pdf>. Acesso em: 15 dez. 2024.